

Cordwell On-Call: The 02:00 Monday Pager

Team discussion exercise for Week 7 Module 03 (ML Pipelines, Automation and API Design)

The Setup

You are tonight's ML platform on-call rotation at Cordwell Home and Hardware. The product review summarizer retrains every Monday at 02:00 through the Airflow DAG you saw in the slides. It is Monday. It is 02:00. Your phone is going off.

Over the next hour, five pages arrive. For each page your team will do three things, in order:

1. Commit. Write your answer before the reveal. No changing it afterward.
2. Argue. The Skeptic must make the strongest case against the Engineer's first answer before the team commits.
3. Fix. Write one sentence describing the change you would make to the DAG.

The instructor reveals the answer after every page. Use the slide deck freely. Nothing here requires code you have not already seen.

Roles (rotate after every page)

Role	What you do	How you win
Navigator	Reads the page aloud, watches the clock, writes the team's committed answer.	The team commits on time, every page.
Engineer	Proposes the first diagnosis and the fix. Points at the slide that supports it.	Your first answer survives the Skeptic and the reveal.
Skeptic	Argues the strongest opposing case before the team commits. Not allowed to say "I agree" until the argument has been made.	You change the team's answer, and the new answer is correct.

Rotation: after Page 1 the Navigator becomes Engineer, the Engineer becomes Skeptic, the Skeptic becomes Navigator. Everyone plays every role at least once.

Page 1: "The DAG is not in the UI"

The previous team lead wrote the retrain DAG last spring, before the Airflow 3 upgrade. It was never redeployed after the upgrade. Tonight was supposed to be its first run on 3.3.1. The dashboard shows no such DAG. Here is the file.

```
from datetime import datetime, timedelta
from airflow.sdk import DAG
from airflow.providers.standard.operators.bash import BashOperator

with DAG(
    dag_id="cordwell_summarizer_weekly_retrain",
    schedule_interval="0 2 * * 1",
    start_date=datetime(2026, 9, 7),
    catchup=False,
    default_args={"sla": timedelta(hours=4)},
) as dag:
    train = BashOperator(
        task_id="train",
        bash_command=(
            "cd /workspace/cordwell && "
            "python train.py --run-name weekly_{{ execution_date | ds }}"
        ),
    )
```

Commit before the reveal: This file contains three Airflow 2 leftovers. How many of them hard crash on Airflow 3.3.1, and how many silently do nothing? Write the two numbers.

Team commit (crash count, silent count):

Then discuss:

- Name each leftover and say what happens: TypeError at parse, UndefinedError at task runtime, or a warning that is discarded.
- The three problems do not surface at the same moment. Which one do you have to fix before you can even see the other two? Why does that make the silent one the most dangerous?
- Where in the Airflow 3 UI or logs would each of the three show up? (Slide: Debugging a Failed Run, and Common Pitfalls and Fixes.)

One-sentence fix:

Page 2: "Which Monday is this?"

You fixed Page 1 and redeployed. Now the scheduling questions start. The DAG header is exactly this:

```
with DAG(
    dag_id="cordwell_summarizer_weekly_retrain",
    schedule="0 2 * * 1",
    start_date=datetime(2026, 9, 7),    # a Monday
    catchup=False,
) as dag:
    ...
```

The train task tags its MLflow run with the rendered value of `{{ data_interval_end | ds }}`. Commit to all six answers before the reveal.

#	Question	Team commit
1	Wall-clock moment the first DagRun actually starts.	
2	logical_date of that first run.	
3	The string the MLflow run gets tagged with on that first run.	
4	A colleague swaps in <code>CronDataIntervalTimetable("0 2 * * 1")</code> with everything else unchanged. Wall-clock start and logical_date of the first run now?	
5	The DAG is paused on Monday Sep 21 at 09:00 (after that morning's run) and unpaused on Wednesday Oct 21 at 09:00. How many runs fire on unpause with <code>catchup=False</code> ? With <code>catchup=True</code> ?	
6	Bonus: the same DAG is later switched to <code>schedule=[reviews]</code> (an Asset). On an asset-triggered run, what is logical_date?	

Then discuss:

- The deck says a plain cron string on Airflow 3 gives a zero-width data interval. Explain to your Skeptic, in one sentence, why that makes the tag from question 3 the date of the run rather than the date of the week it summarizes.
- Which of the two timetables would you actually want for a weekly retrain on last week's reviews? What is the cost of picking the wrong one and not noticing for a month?

One-sentence fix (which timetable, and why):

Page 3: "The champion is worse than last week"

Different Monday. The run finished green. At 08:30 a merchandising analyst reports that this morning's review summaries read like they were written by a snow blower. You pull the MLflow numbers: the model now serving as @champion scored rouge_l 0.36. ROUGE_MIN is 0.38. The gate should have caught it. Here is the relevant part of the DAG after a "small refactor" someone shipped last week.

```
@task
def train(**context) -> dict:
    with mlflow.start_run(run_name=f"weekly_{context['run_id']}") as run:
        # ... training ...
        mlflow.transformers.log_model(transformers_model=pipe, name="review_summarizer")
        mv = mlflow.register_model(f"runs/{run.info.run_id}/review_summarizer", MODEL_NAME)
        MlflowClient().set_registered_model_alias(MODEL_NAME, "challenger", mv.version)
        return {"mlflow_run_id": run.info.run_id, "version": mv.version}

@task
def evaluate(train_out: dict) -> dict:
    # ... returns {"rouge_l": ..., "toxicity_rate": ...}

@task.branch
def quality_gate(metrics: dict) -> str:
    if metrics["rouge_l"] >= ROUGE_MIN and metrics["toxicity_rate"] <= TOX_MAX:
        return "log_pass"
    return "alert_quality_regression"

@task
def log_pass():
    print("quality gate passed")

@task
def alert_quality_regression():
    print("quality gate FAILED; paging ML platform")

@task(trigger_rule="none_failed")
def promote_challenger() -> int:
    c = MlflowClient()
    v = c.get_model_version_by_alias(MODEL_NAME, "challenger").version
    c.set_registered_model_alias(MODEL_NAME, "champion", v)
    return v

train_out = train()
metrics = evaluate(train_out)
gate = quality_gate(metrics)
gate >> [log_pass(), alert_quality_regression()] >> promote_challenger()
```

Commit before the reveal: The gate correctly returned "alert_quality_regression". Did promote_challenger run? Yes or no.

Team commit (yes or no):

Then discuss:

- There are two separate mistakes here that had to combine to promote a bad model. One is about where a line of code lives. The other is about a trigger rule. Find both.
- The alert task printed its message and exited 0. From Airflow's point of view, was that a success? What does "none_failed" think about a skipped sibling?

- It is 08:35 and the analyst is waiting. What is the single MLflow call that gets the previous model serving again? (Slide: Promotion and Rollback with Aliases.) Does it require a redeploy?
- The deck says promotion to @champion should be "a separate, deliberate task." Should promote_challenger be in this DAG at all, or should a human press the button? Skeptic, take the side the Engineer did not.

One-sentence fix (what moves, what changes):

Page 4: "Nobody got paged"

Two related pages arrive together. First, the metadata database has grown from 40 MB to 6 GB in five weeks and the API server is timing out when the UI opens the XCom tab. Second, the on-call Slack channel has been silent for a month even though three task failures show in the grid view. Here are the two snippets involved.

```
@task
def train(**context) -> dict:
    # ... training ...
    return {
        "weights": {k: v.tolist() for k, v in model.state_dict().items()},
        "tokenizer_vocab": tokenizer.get_vocab(),
        "mlflow_run_id": run.info.run_id,
    }

ALERT_MODE = "offline"    # switch to "webhook" in production

def alert_on_failure(context):
    ti = context["task_instance"]
    msg = f"[Cordwell retrain] task {ti.task_id} failed (try {ti.try_number})"
    if ALERT_MODE == "offline":
        print(msg)
        return
    webhook = BaseHook.get_connection("cordwell_alerts").password
    requests.post(webhook, json={"text": msg}, timeout=10).raise_for_status()
```

Commit before the reveal: Which rule from the XComs slide was broken, and where did the three failure messages actually go?

Team commit:

Then discuss:

- Rewrite the train return value in your head. What should cross XCom, and where does everything else live? (Slide: XCom for Model Paths.)
- ALERT_MODE is a module constant in the DAG file. What did the deck say config should be instead, and what would have made this impossible to forget at deploy time?
- The three failures were on pull_reviews, each retried three times before failing for good. The error was an expired DVC remote credential. Was retrying the right call? Which list on the Retries and Backoff slide does that error belong to?

One-sentence fix:

Page 5: "The annotations land whenever they land"

Last page, and this one is a design argument rather than a bug. The human annotation vendor drops `weekly_annotations_*.jsonl` into the incoming folder sometime between Sunday 18:00 and Monday 06:00. Nobody can pin down when. The retrain must not start without it, and it must not wait so long that the Monday 08:00 merchandising refresh runs on last week's model. Four proposals are on the table.

Option	Proposal	Champion
A	Move the cron to 06:00 Monday and trust the vendor to be done by then.	Skeptic (assigned)
B	Keep 02:00. Add a FileSensor in poke mode, <code>poke_interval=30</code> , timeout of four hours.	
C	Keep 02:00. Add a FileSensor in reschedule mode, <code>poke_interval=300</code> , timeout of four hours.	
D	Drop the cron. A small ingest DAG watches the drop and declares the file as an Asset outlet; the retrain DAG uses <code>schedule=[annotations]</code> .	

Commit before the reveal: Pick one option. Then write one real downside of the option you picked. A pick with no stated downside scores zero.

Team commit (option and its downside):

Then discuss:

- For option B, what is the worker slot doing at 03:47? For option C? For option D? (Slide: Sensors, poke vs reschedule.)
- Option D removes the clock entirely. What does that do to `logical_date`, and what breaks in a task that still reads it? What still needs a deadline?
- The Skeptic must defend option A for one full minute. What real-world reason might a team choose it anyway? (Hint: the deck calls cron "a deliberate choice, not a law.")

One-sentence fix (what you would ship, and the one thing you would add to it):

Debrief (5 minutes)

Each team shares one thing with the room:

- The page where your committed answer was wrong, and the sentence from the deck that would have saved you.
- The best Skeptic argument you heard, even if it lost.

Why this matters at Cordwell

Every page tonight was a real failure mode from the slides: legacy syntax that crashes or silently vanishes, a schedule that means something different than it reads, a gate that a trigger rule walked around, a config constant that never got flipped, and a wait that pins a worker. None of them needed new code to fix. They needed someone in the room who had committed to an answer and been wrong once already. That is you, now.